```
#CAR PRICE PREDICTION PROJECT
#AUTHOR: PARNITA
#Internship: INTERNPE
#Submitted on: 28 June 2025



# Step 1: Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import mean_squared_error, r2_score
import pickle
```

```
# Step 2: Load the Dataset
df = pd.read_csv('/content/quikr_car.csv')
df.head()
```

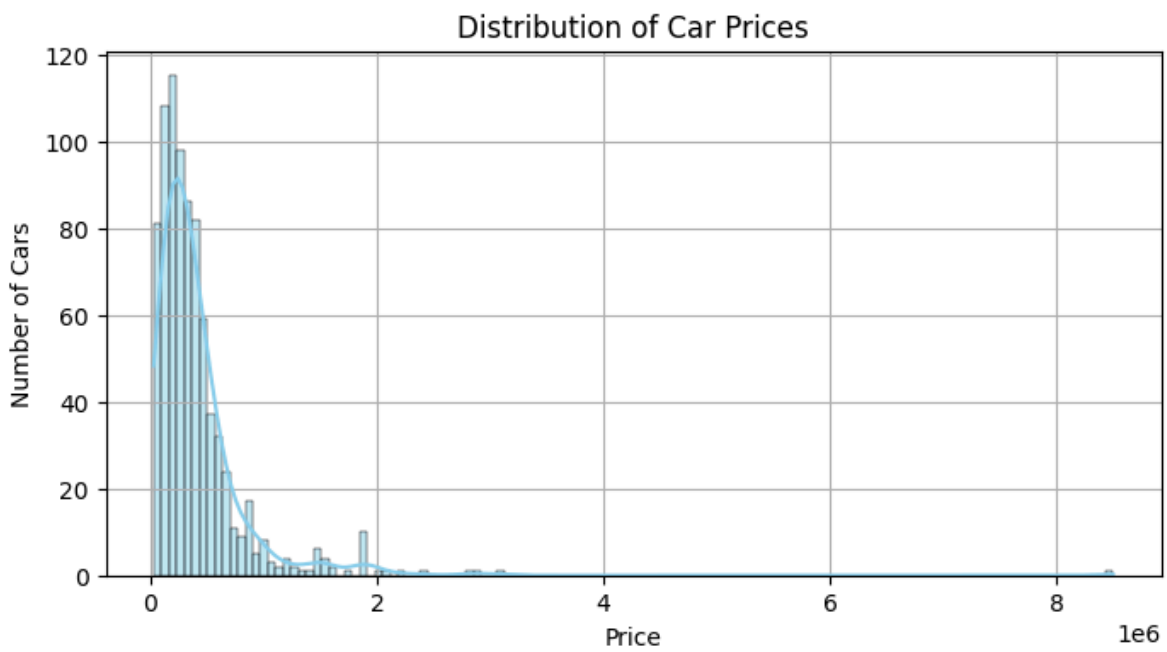
	name	company	year	Price	kms_driven	fuel_type
0	Hyundai Santro Xing XO eRLX Euro III	Hyundai	2007	80,000	45,000 kms	Petrol
1	Mahindra Jeep CL550 MDI	Mahindra	2006	4,25,000	40 kms	Diesel
2	Maruti Suzuki Alto 800 Vxi	Maruti	2018	Ask For Price	22,000 kms	Petrol
3	Hyundai Grand i10 Magna 1.2 Kappa VTVT	Hyundai	2014	3,25,000	28,000 kms	Petrol
4	Ford EcoSport Titanium 1.5L TDCi	Ford	2014	5,75,000	36,000 kms	Diesel

```
 # Step 3: Clean the Data Safely

# Convert Price column to string to safely use .str functions
df['Price'] = df['Price'].astype(str)

# Remove rows where price is 'Ask For Price'
df = df[~df['Price'].str.contains('Ask', na=False)].copy()
```

```
# Remove ₹ symbol and commas, then convert to integers
df['Price'] = df['Price'].str.replace('₹', '', regex=False)
df['Price'] = df['Price'].str.replace(',', '', regex=False)
df['Price'] = df['Price'].astype(int)

# Drop any remaining null values
df.dropna(inplace=True)
df.reset_index(drop=True, inplace=True)
```

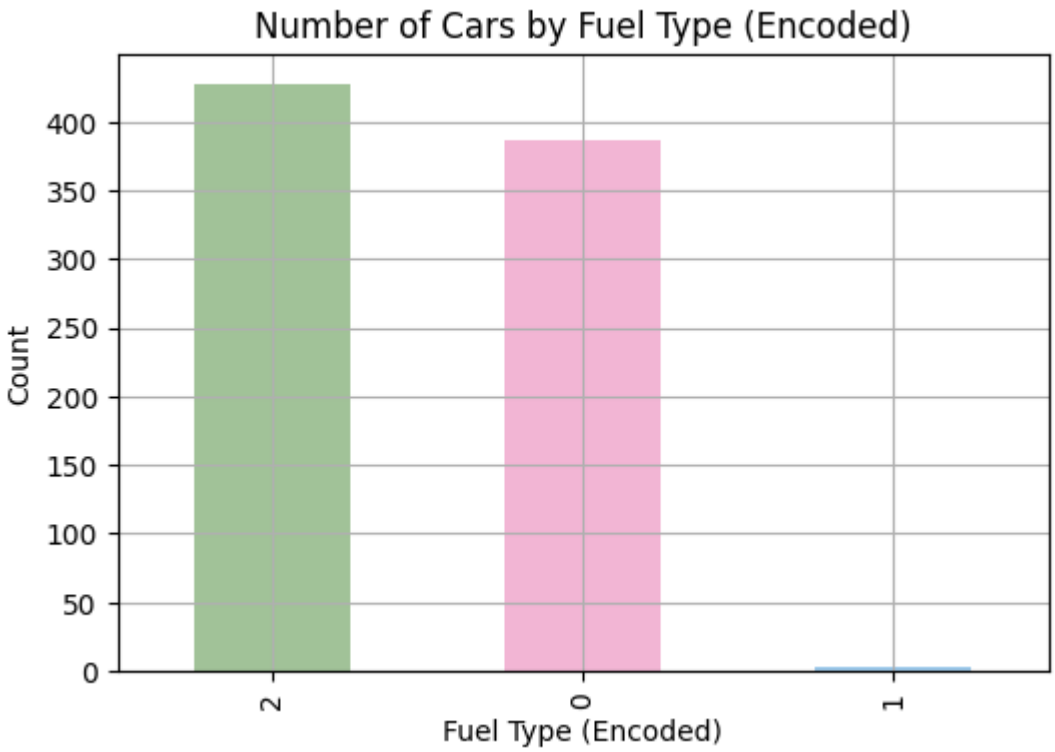
```
# Step 4: Encode Categorical Columns
le = LabelEncoder()
categorical_cols = df.select_dtypes(include=['object']).columns

for col in categorical_cols:
    df[col] = le.fit_transform(df[col])
```

```
# Step 4.1: Distribution of Car Prices       (This Shows how prices are spre
plt.figure(figsize=(8,4))
sns.histplot(df['Price'], kde=True, color='skyblue')
plt.title("Distribution of Car Prices")
plt.xlabel("Price")
plt.ylabel("Number of Cars")
plt.grid(True)
plt.show()
```




```
# Step 4.2: Count of Cars by Encoded Fuel Type          (This gives cate
plt.figure(figsize=(6,4))
df['fuel_type'].value_counts().plot(kind='bar', color=['#A1C298', '#F2B5D
```

```
plt.title("Number of Cars by Fuel Type (Encoded)")
plt.xlabel("Fuel Type (Encoded)")
plt.ylabel("Count")
plt.grid(True)
plt.show()
```




```
# Step 4.3: Feature Correlation with Price          (This Shows which numer
plt.figure(figsize=(10,5))
sns.heatmap(df.corr()[['Price']].sort_values(by='Price', ascending=False), a
plt.title("Feature Correlation with Price")
plt.show()
```

```
# Step 5: Define Input Features and Target
X = df.drop(['Price'], axis=1)
y = df['Price']
```

```
# Step 6: Split the Data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran
print("Training Set Shape:", X_train.shape)
print("Testing Set Shape:", X_test.shape)
```

```
Training Set Shape: (652, 5)
Testing Set Shape: (164, 5)
```

```
# Step 7: Train the Model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict on test set
y_pred = model.predict(X_test)
```

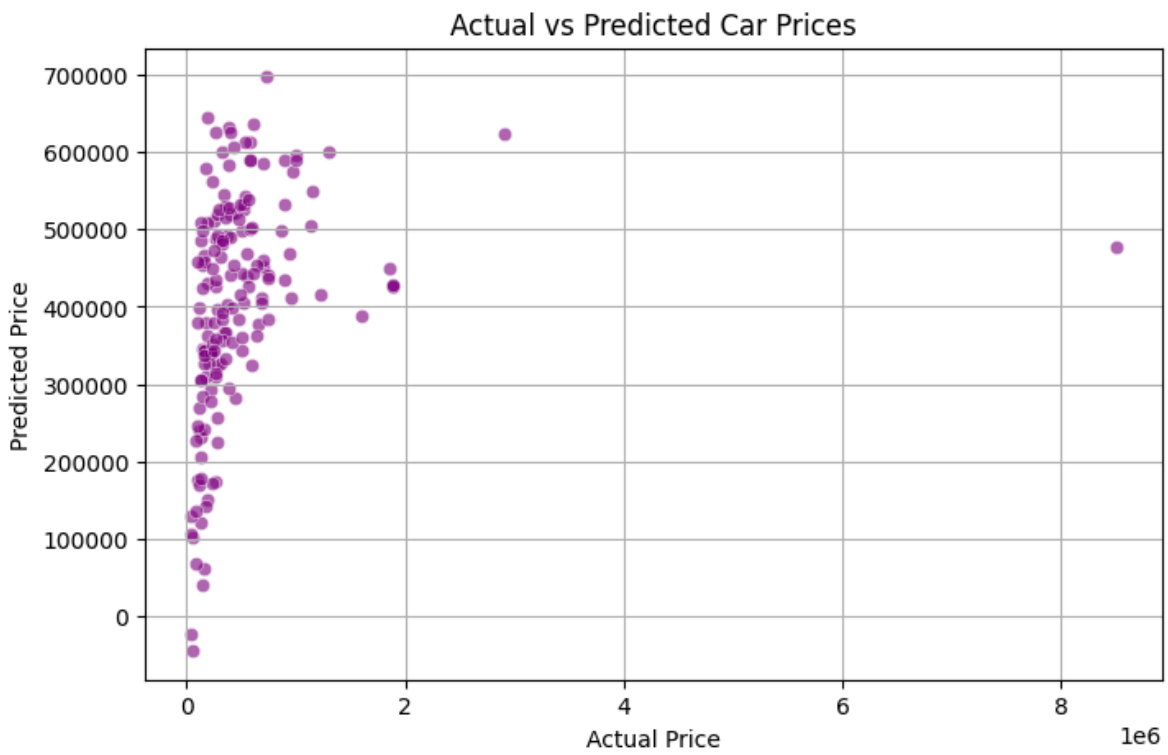
```
# Step 7.1: Predict Price for a New Car (Example)
sample = X_test.iloc[0:1]
predicted_price = model.predict(sample)
print("Predicted Price for Sample Car:", round(predicted_price[0], 2))
```

```
Predicted Price for Sample Car: 206022.76
```

```
# Step 8: Model Evaluation
r2 = r2_score(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print("R2 Score:", round(r2, 3))
print("Root Mean Squared Error (RMSE):", round(rmse, 2))
```

```
R2 Score: 0.047
Root Mean Squared Error (RMSE): 730085.47
```

```
# Step 9: Visualize Actual vs Predicted Prices
plt.figure(figsize=(8,5))
sns.scatterplot(x=y_test, y=y_pred, color='purple', alpha=0.6)
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted Car Prices")
plt.grid(True)
plt.show()
```




```
# Step 10: Save the Model
pickle.dump(model, open('car_price_lr_model.pkl', 'wb'))
print("Model saved as car_price_lr_model.pkl")
```

```
Model saved as car_price_lr_model.pkl
```